Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

MY SCREENSHOTS

Task_3:

1. Created a folder for week 3 practical

```
ashborn@DESKTOP-I4G5Q4N:~$ mkdir week3_p
ashborn@DESKTOP-I4G5Q4N:~$ cd week3_p
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ pwd
/home/ashborn/week3_p
```

2. Created a C-program and compiled it

```
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ nano fork_process.c
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ gcc fork_process.c -o fork_process
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ls
fork_process  fork_process.c
ashborn@DESKTOP-I4G5Q4N:~/week3_p$
```

Source Code –

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
   pid_t pid;
```

```
    printf("Before fork():\n");
    printf("PID  = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());
    pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    if (pid == 0)
    {
        // Child process
        printf("CHILD PROCESS\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child is running...\n");
        sleep(5);
        printf("Child process terminating.\n");
    }
    else
    {
        // Parent process
        printf("PARENT PROCESS\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        printf("Child PID = %d\n", pid);
        printf("Parent is waiting for child...\n");
        wait(NULL);
        printf("Child has terminated.\n");
        printf("Parent process terminating.\n");
    }
    return 0;
}
```

3. Executed the program

```
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ./fork_process
Before fork():
PID  = 71
PPID = 11

PARENT PROCESS
CHILD PROCESS
PID  = 71
PID  = 72
PPID = 11
PPID = 71
Child PID = 72
Child is running...
Parent is waiting for child...
Child process terminating.
Child has terminated.
Parent process terminating.
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
ashborn     11    10  0 04:37 tty1     00:00:00 -bash
ashborn     73    11  0 04:48 tty1     00:00:00 ps -f
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ps -ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 04:37 ?        00:00:00 /init
root         7     1  0 04:37 ?        00:00:00 plan9 --control-socket 6 --log-level 4
root        10     1  0 04:37 tty1     00:00:00 /init
ashborn     11    10  0 04:37 tty1     00:00:00 -bash
ashborn     74    11 99 04:49 tty1     00:00:00 ps -ef
ashborn@DESKTOP-I4G5Q4N:~/week3_p$
```

Summary –

A C program was developed using the fork() system call to create a parent and child process. The PID and PPID of both processes were displayed, and wait() was used to make the parent wait for the child process. The processes were then monitored using Linux commands such as ps, /proc, and top to understand their execution and states.

TASK_4:

1. Replacing the previous C-program's code
   Source Code –

ashborn@DESKTOP-I4G5Q4N: ~/week3_p

GNU nano 7.2

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent starting...\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nCHILD PROCESS\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());

        printf("Child is RUNNING...\n");

        sleep(10);

        printf("Child terminating...\n");
        sleep(2);
    }
    else
    {
        printf("\nPARENT PROCESS\n");
        printf("PID       = %d\n", getpid());
        printf("PPID      = %d\n", getppid());
        printf("Child PID = %d\n", pid);

        printf("Parent is WAITING for child...\n");

        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent terminating...\n");
    }

    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut
^X Exit ^R Read File ^\ Replace ^U Paste

2. Executed the program

```
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ./fork_process
Parent starting...


PARENT PROCESS
CHILD PROCESS
PID      = 98
PID  = 99
PPID     = 11
PPID = 98
Child PID = 99
Child is RUNNING...
Parent is WAITING for child...
Child terminating...
Child terminated.
Parent terminating...
ashborn@DESKTOP-I4G5Q4N:~/week3_p$
```

```
ashborn@DESKTOP-I4G5Q4N:~$ ps -ef | grep fork_process
ashborn     98    11  0 04:57 tty1     00:00:00 ./fork_process
ashborn     99    98  0 04:57 tty1     00:00:00 ./fork_process
ashborn    101    82  0 04:57 tty2     00:00:00 grep --color=auto fork_process
ashborn@DESKTOP-I4G5Q4N:~$
```

3. Observing the execution through

```
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ./fork_process
Parent starting...

PARENT PROCESS

PID      = 102
CHILD PROCESS
PPID     = 11
PID  = 103
Child PID = 103
PPID = 102
Parent is WAITING for child...
Child is RUNNING...
Child terminating...
Child terminated.
Parent terminating...
ashborn@DESKTOP-I4G5Q4N:~/week3_p$
```

```
ashborn@DESKTOP-I4G5Q4N:~$ ps -o pid,ppid,state,stat,cmd -C fork_process
  PID  PPID S STAT CMD
  102    11 S S    ./fork_process
  103   102 S S    ./fork_process
ashborn@DESKTOP-I4G5Q4N:~$
```

4. Observing the execution through

```
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ./fork_process
Parent starting...


PARENT PROCESS
CHILD PROCESS
PID      = 108
PID  = 109
PPID     = 11
PPID = 108
Child PID = 109
Child is RUNNING...
Parent is WAITING for child...
Child terminating...
Child terminated.
Parent terminating...
ashborn@DESKTOP-I4G5Q4N:~/week3_p$
```

```
ashborn@DESKTOP-I4G5Q4N:~$ grep -E "Name|State|Pid|PPid" /proc/109/status
Name:   fork_process
State:  S (sleeping)
Pid:    109
PPid:   108
TracerPid:      0
ashborn@DESKTOP-I4G5Q4N:~$
```

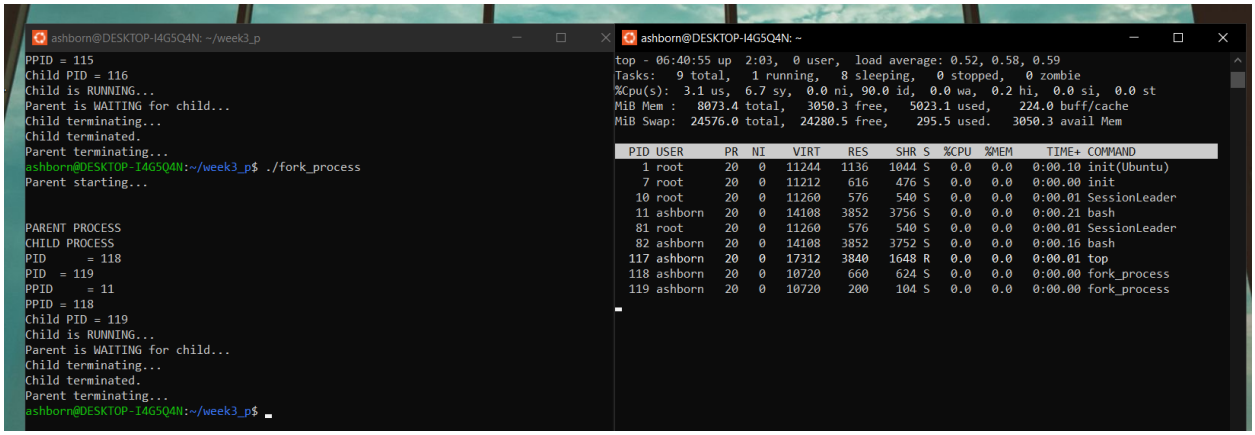
5. Observing the execution through ‘top’ command

```
ashborn@DESKTOP-I4G5Q4N:~/week3_p$ ./fork_process
Parent starting...


PARENT PROCESS
CHILD PROCESS
PID       = 111
PID   = 112
PPID      = 11
PPID = 111
Child PID = 112
Child is RUNNING...
Parent is WAITING for child...
Child terminating...
Child terminated.
Parent terminating...
ashborn@DESKTOP-I4G5Q4N:~/week3_p$
```

```
ashborn@DESKTOP-I4G5Q4N:~$ top
top - 05:11:34 up 34 min,  0 user,  load average: 0.52, 0.58, 0.59
Tasks:   7 total,   1 running,   6 sleeping,   0 stopped,   0 zombie
%Cpu(s):  7.1 us,  8.7 sy,  0.0 ni, 83.7 id,  0.0 wa,  0.5 hi,  0.0 si,  0.0 st
MiB Mem :   8073.4 total,   3010.0 free,   5063.4 used,    224.0 buff/cache
MiB Swap:  24576.0 total,  24302.6 free,    273.4 used.   3010.0 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root      20   0   11244   1140   1060 S   0.0   0.0   0:00.09 init(Ubuntu)
    7 root      20   0   11212    620    480 S   0.0   0.0   0:00.00 init
   10 root      20   0   11260    576    524 S   0.0   0.0   0:00.01 SessionLeader
   11 ashborn   20   0   14108   3852   3588 S   0.0   0.0   0:00.20 bash
   81 root      20   0   11260    576    524 S   0.0   0.0   0:00.01 SessionLeader
   82 ashborn   20   0   14108   3852   3748 S   0.0   0.0   0:00.16 bash
  113 ashborn   20   0   17312   3840   1644 R   0.0   0.0   0:00.01 top
```



Observation:

The ps, top, and /proc commands were used to monitor the parent and child processes. The process PID, PPID, and state were observed during execution. The parent process entered a waiting state while waiting for the child, and the child process changed from running to terminated. After termination, both processes disappeared from the process list.